Архитектурные проблемы классических LLM и их решение

Елена Кантонистова

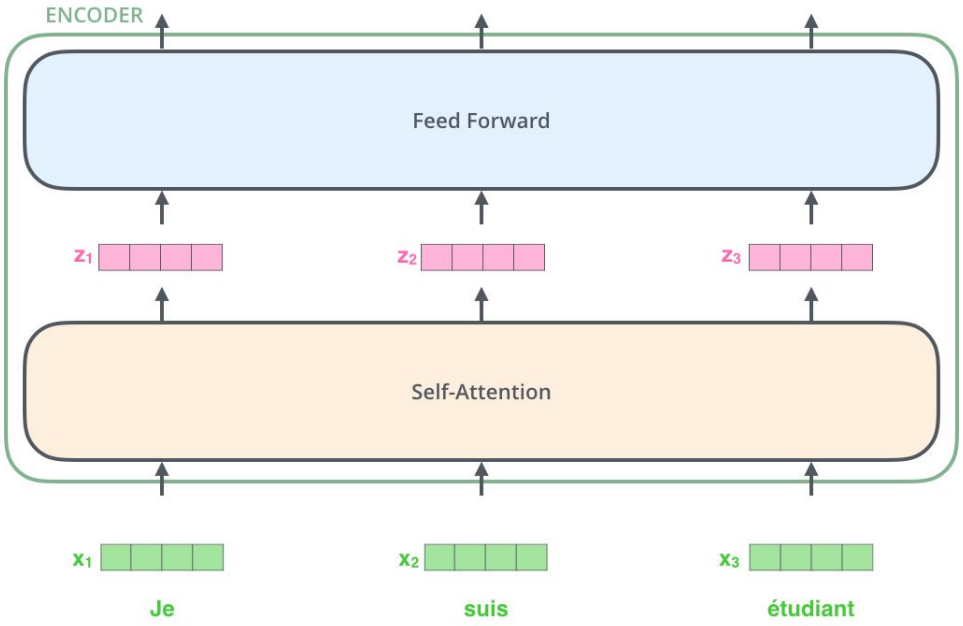
План

- Вспоминаем трансформер
- Модификации positional embeddings
- Модификации BERT и сравнение bert-like моделей
- Mixture of Experts

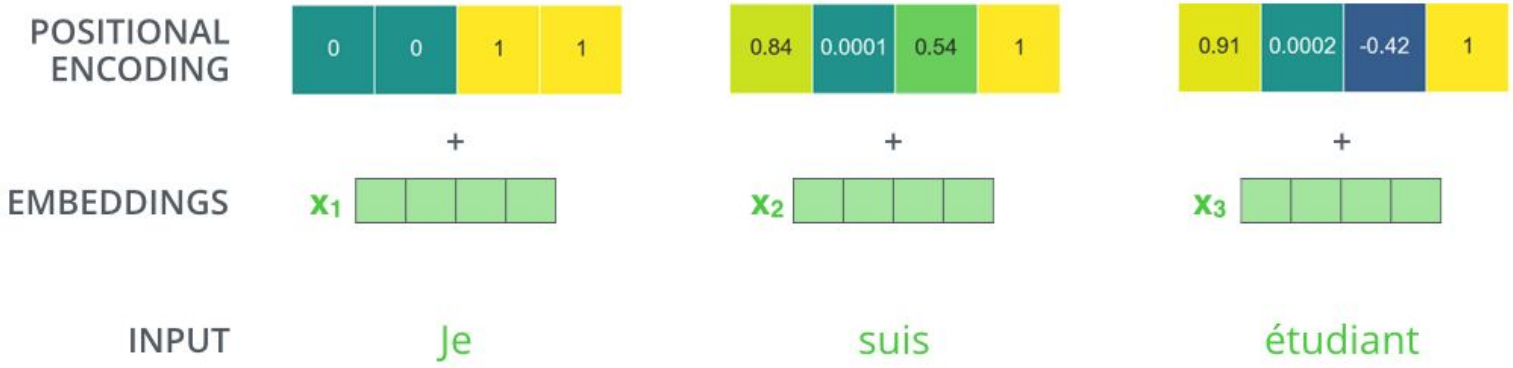
Transformer Recap

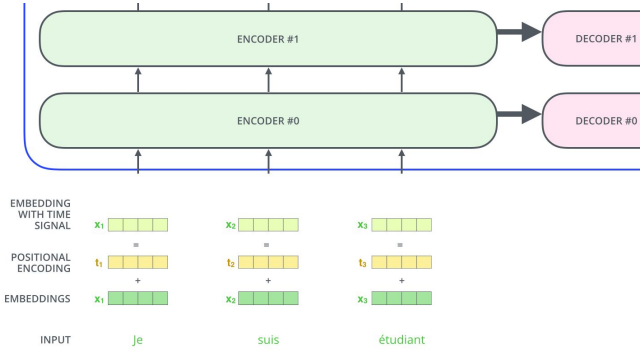
Attention: матричный вид

- На вход подается текст некоторой длины max_length
- Каждый токен закодирован своим id
- При проходе через embedding-слой токен становится вектором фиксированной длины (например, 512) со встроенным знанием позиции



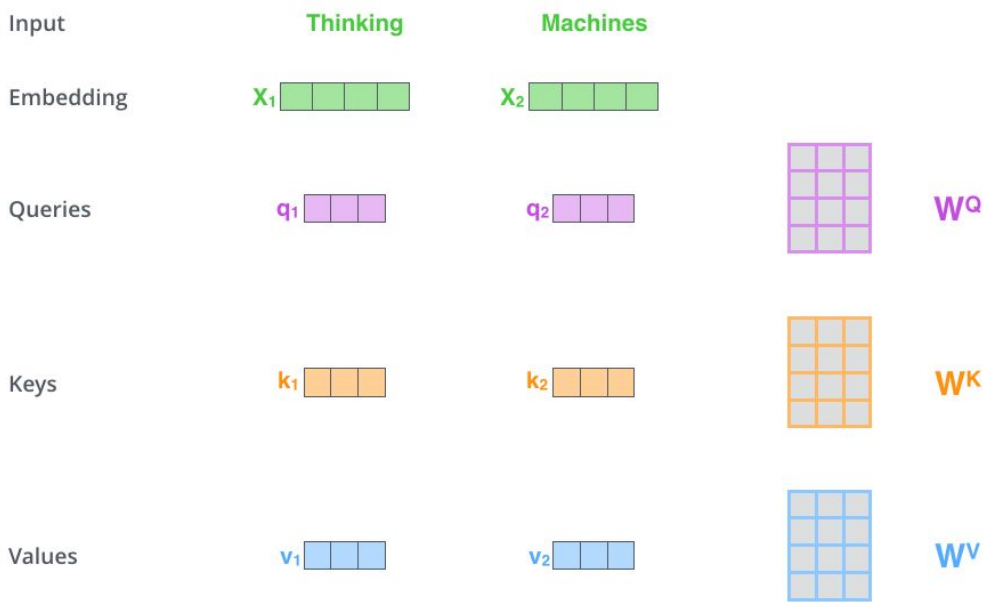
Позиционное кодирование





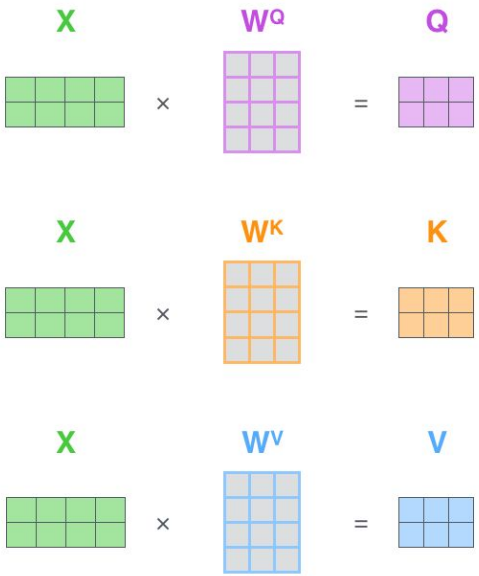
Attention: матричный вид

- На вход в attention передается вектор-эмбеддинг слова
- Далее он умножается на матрицы Wq, Wk, Wv и получаются векторы q, k, v для каждого токена



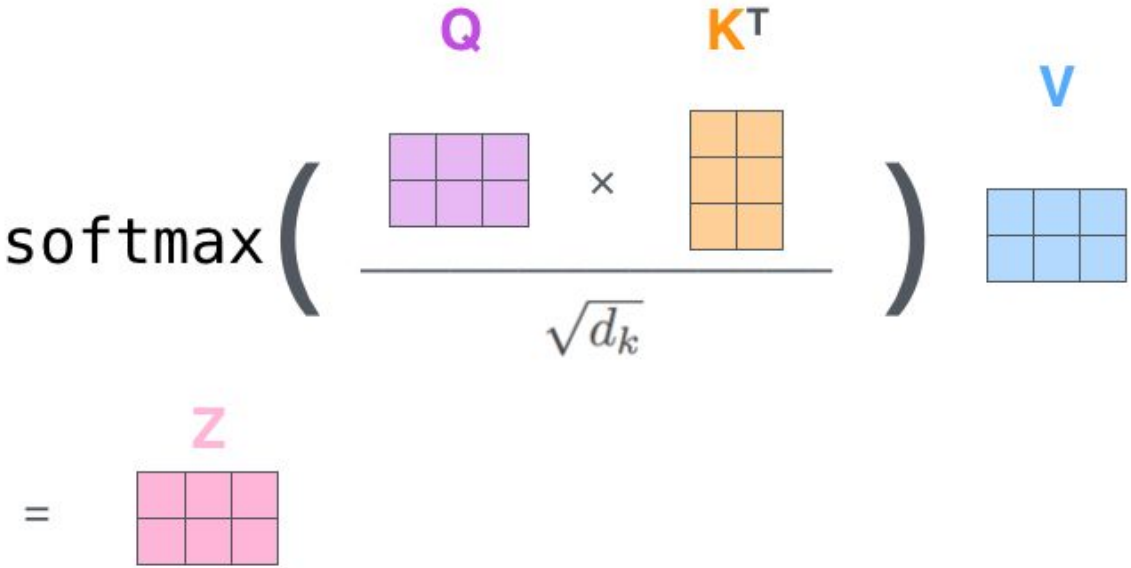
Attention: матричный вид

- Текст, поданный на вход, можно представить в виде матрицы
- Тогда после прохождения через attention-слой мы получим матрицы Q, K, V, где строк столько, сколько было слов в тексте (сколько было строк в зеленой матрице)



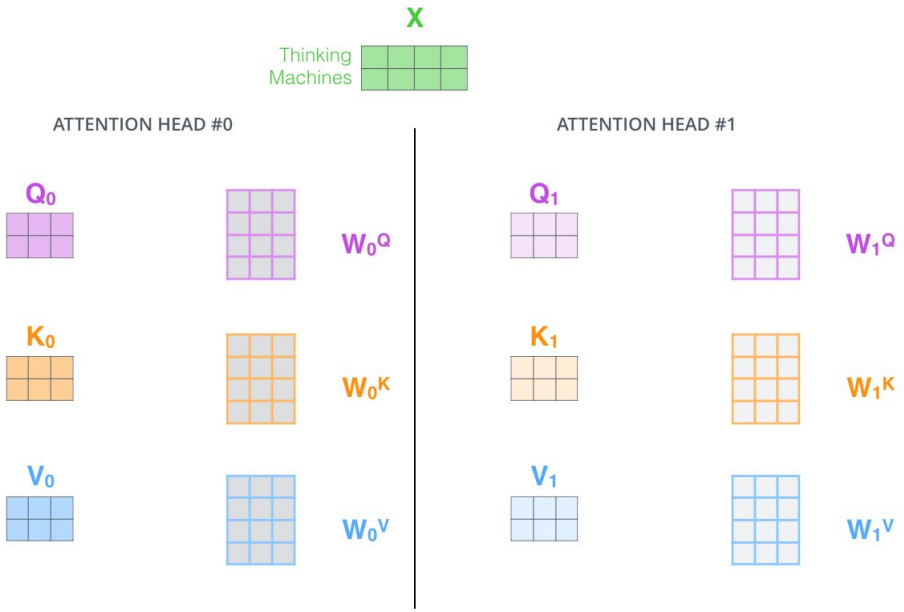
Attention: матричный вид

- Softmax и дальнейшее умножение на values также можно записать в векторном виде



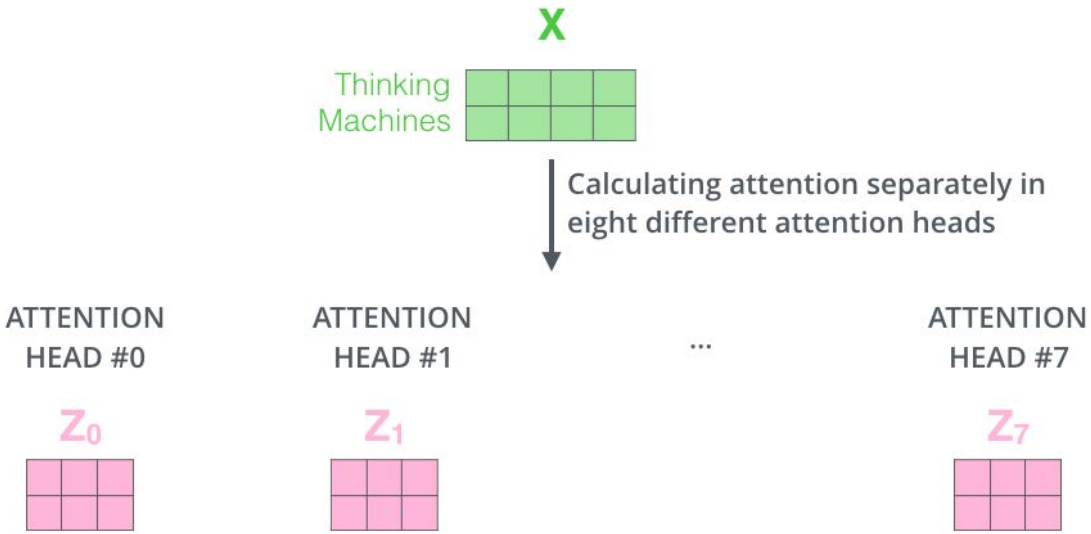
Attention: матричный вид

- Обычно в attention несколько голов, как это можно представить матрично?



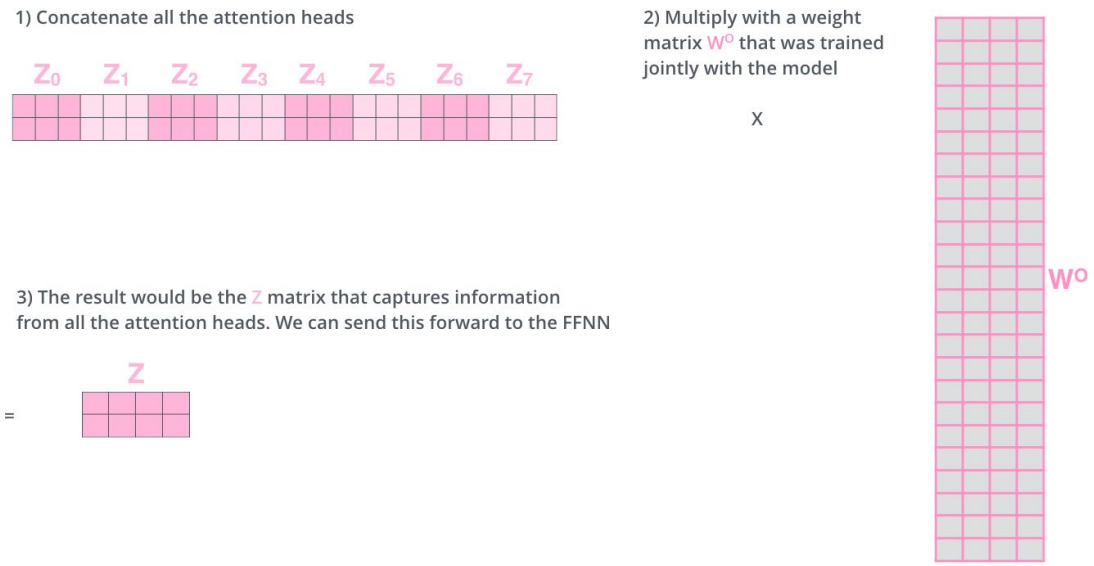
Attention: матричный вид

- Каждая голова attention выдает вектор внимания для каждого слова в последовательности

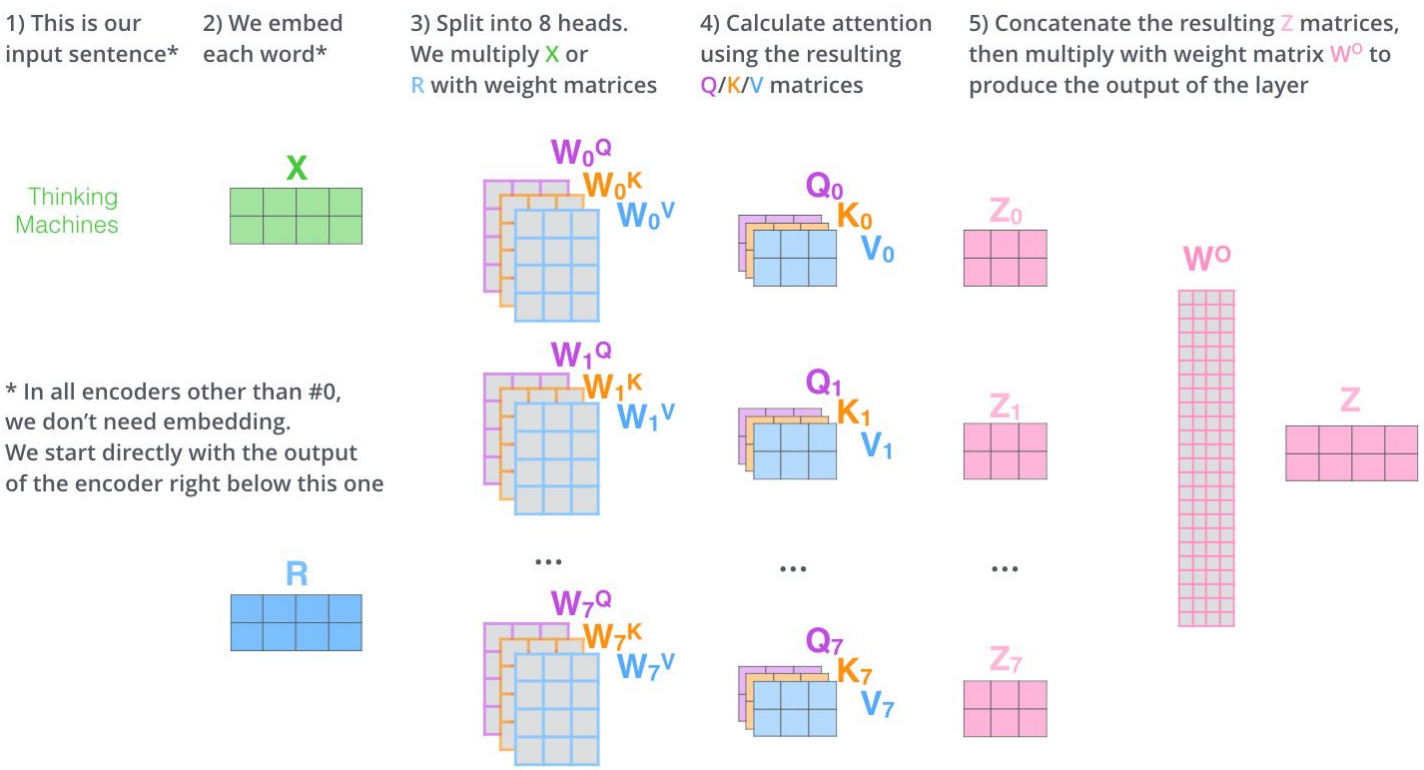


Attention: матричный вид

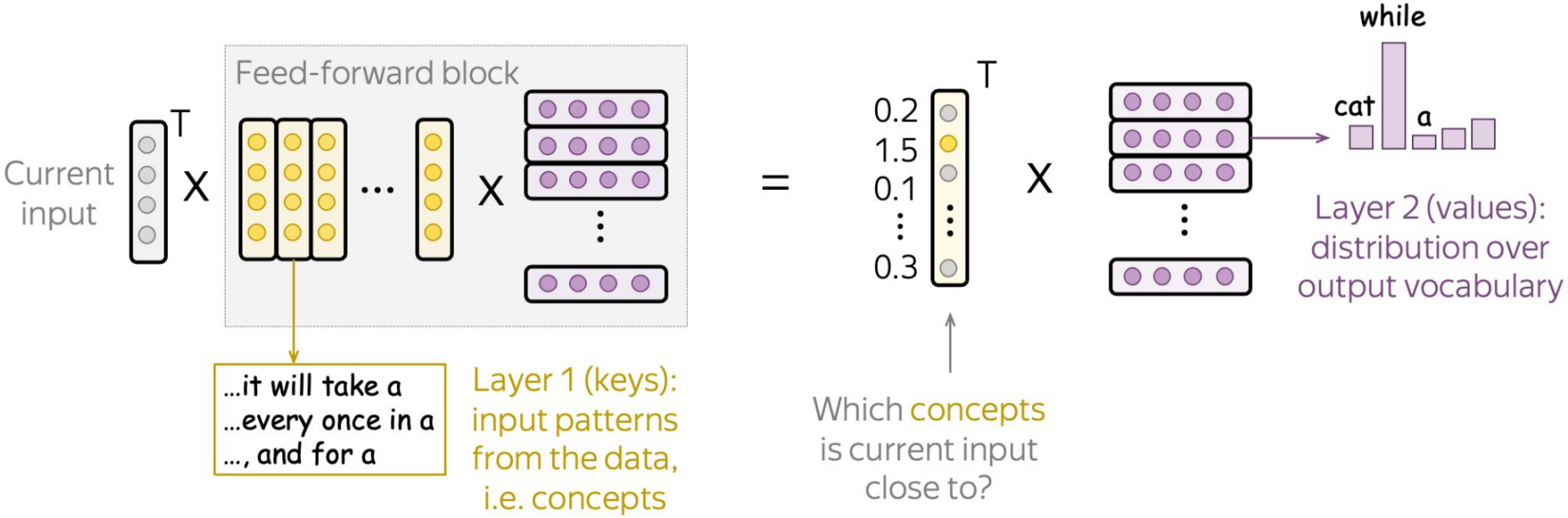
- Затем эти векторы (матрицы) конкатенируются и результат умножается на дополнительную матрицу весов W0 - в итоге получается вектор многоголового внимания для каждого слова из последовательности



Attention: матричный вид



FFN-слои


Feed-forward block
Current input
T
X
...
X
=
0.2
1.5
0.1
0.3
T
X
while
cat
a
...it will take a
...every once in a
..., and for a
Layer 1 (keys): input patterns from the data, i.e. concepts
Which concepts is current input close to?
Layer 2 (values): distribution over output vocabulary

Модификации Positional Encoding

Проблемы positional encoding

1. Плохая обобщаемость на более длинные последовательности

- Абсолютные позиционные эмбеддинги обычно **ограничены максимальной длиной, на которую модель обучалась** (например, 512 или 1024 токена).
- Если на этапе инференса приходит текст длиннее, чем максимальная позиция. Мы можем закодировать эту позицию формулами, но в целом таких частот синуса и косинуса модель в обучении не видела и может неправильно обобщить.
- Это ведёт к снижению качества обработки очень длинных текстов.

2. Жёсткая зависимость от позиции

- Абсолютные эмбеддинги задают позицию жёстко и независимо от соседних токенов.
- Модель учится «что позиция 10 — это позиция 10», но не учитывает, что важна не только абсолютная позиция, но и **относительное положение токенов друг к другу**.
- Относительные позиции лучше отражают смысловые связи (например, «следующий токен» или «токен через 3 слова»).

Relative Attention

Relative attention (относительное внимание) – это модификация механизма внимания (self-attention), которая вместо *абсолютных позиций* токенов (1, 2, 3, ...) учитывает **расстояние между токенами**.

Идея в том, что для понимания текста важнее *относительные позиции* (например, что слово B идёт через 2 позиции после слова A), чем абсолютное место в последовательности (позиция 27).

$$\mathrm{RelativeAttention} = \mathrm{Softmax}\left(\frac{QK^\top + S_{rel}}{\sqrt{D_h}}\right)V$$

Relative Attention

S_rel[i,j] = Q_i · R_(j−i)

- `S_rel[i,j]` — это вклад относительной позиции в attention-скор между токенами `i` и `j`.

Он показывает, насколько важно расстояние `j−i` именно для токена `i`.

- R_(j-i) - это эмбеддинг разности j-i, то есть эмбеддинг расстояния j-i, который не зависит от конкретного токена

$$\mathrm{RelativeAttention} = \mathrm{Softmax}\left(\frac{QK^\top + S_{rel}}{\sqrt{D_h}}\right)V$$

Rotary attention

RoPE предлагает другой путь:

- Мы кодируем позицию токена через **поворот (rotation)** его вектора эмбеддинга на угол, зависящий от позиции:

  каждому токену с позицией p применяется поворот вектора на угол, зависящий от p.

- При вычислении QK^T внимание автоматически учитывает относительное смещение токенов, потому что разница углов вращений кодирует расстояние между позициями. То есть для внимания абсолютные позиции в итоге не учитываются, а учитывается только разность между ними:

$$\boldsymbol{q}_m^\intercal \boldsymbol{k}_n = (\boldsymbol{R}_{\Theta,m}^d \boldsymbol{W}_q \boldsymbol{x}_m)^\intercal (\boldsymbol{R}_{\Theta,n}^d \boldsymbol{W}_k \boldsymbol{x}_n) = \boldsymbol{x}^\intercal \boldsymbol{W}_q R_{\Theta,n-m}^d \boldsymbol{W}_k \boldsymbol{x}_n$$

Rotary attention

RoPE предлагает другой путь:

- Мы кодируем позицию токена через **поворот (rotation)** его вектора эмбеддинга на угол, зависящий от позиции.
- При вычислении QK^T внимание автоматически учитывает относительное смещение токенов, потому что разница углов вращений кодирует расстояние между позициями.

$$\boldsymbol{R}^d_{\Theta,m} = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

$$\theta_{p,j} = p \cdot \omega_j = p \cdot \frac{1}{\mathrm{base}^{j/d}}$$

ALiBi

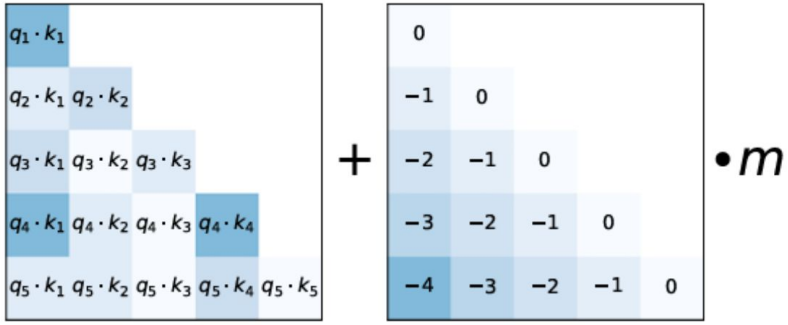
В обычном self-attention считаем:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

ALiBi добавляет **линейный штраф** для пар токенов, которые далеко друг от друга:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + B\right)V$$

где $B_{ij} = -m \cdot |i - j|$,
а m — наклон (slope), задающий, насколько сильно внимание уменьшается с увеличением расстояния.



m is a constant with **one value per head**:

$$\mathrm{m}_i = \frac{1}{2^{\mathrm{i} \cdot \text{scale}}}, \text{scale=0.5 (for example)}$$

Сравнение подходов

Метод	Тип кодирования	Зависимость от позиции	Обучаемость
Синусоидальные PE	Абсолютная (формульная)	Да (абсолютная)	Нет (фиксированная формула)
Relative Attention (Shaw et al.)	Относительная	Да (через R_{j-i})	Да (обучаемые R_{j-i})
RoPE (Rotary Positional Embedding)	Абсолютная → через вращение вектора	Да (через угол)	Нет (повороты фиксированы)
ALiBi (Attention Linear Biases)	Относительная (линейный bias)	Да (через расстояние j-i)	Нет

Сравнение подходов

Метод	Обобщаемость на длинные тексты	Плюсы	Минусы
Синусоидальные PE	Средняя; плохо, если выходить за диапазон обучения	Простая, не требует параметров; может экстраполировать на чуть большие длины	Абсолютные позиции, нет инвариантности к сдвигу; ограничены длиной обучения
Relative Attention (Shaw et al.)	Хорошая; стабильна на длинных текстах	Относительные расстояния прямо в модели; гибко учитывает разную важность для разных токенов	Нужны обучаемые embeddings для каждой разности;
RoPE (Rotary Positional Embedding)	Очень хорошая; относительные расстояния получаются автоматически	Простой способ встроить относительные позиции; хорошо масштабируется на длинные контексты	Сложнее для интуиции; зависит от размерности вектора; требует тригонометрических операций
ALiBi (Attention Linear Biases)	Отличная; легко	Очень простая, не требует	Грубая аппроксимация;

Использование в моделях

Модель / Архитектура	Тип позиционного кодирования
BERT-like (BERT, RoBERTa, DistilBERT)	Абсолютные позиционные embeddings (learned или синусоидальные) — классический способ для encoder-only моделей.
GPT-like (GPT-4 и другие GPT-серии)	**RoPE (Rotary Positional Embedding)** — современный способ, широко применяемый в decoder-only LLM.
Qwen (Alibaba)	**RoPE** — используется в современных Qwen-моделях с decoder-only архитектурой.
DeepSeek (DeepSeek-V3 / R1)	**RoPE** — также построена на transformer-подобной архитектуре с rotary позициями.
GigaChat и YandexGPT	**RoPE** (скорее всего) — современные GPT-подобные LLM конструктивно учитывают относительные позиции через RoPE.
Transformer-XL	**Relative positional encoding** — заменяет абсолютные позиции на относительные для улучшения длинной зависимости в последовательностях. (scaler.com)

FFN

- Attention-слои выдают нам векторы токенов, в которых учитывается влияние других токенов
- FFN-слои перевзвешивают координаты этих векторов, анализируя уже каждый токен в отдельности

$$\mathrm{FFN}_{\mathrm{GLU}}(x, W, V, W_2) = (\sigma(xW) \otimes xV)W_2$$

$$\mathrm{FFN}_{\mathrm{Bilinear}}(x, W, V, W_2) = (xW \otimes xV)W_2$$

$$\mathrm{FFN}_{\mathrm{ReGLU}}(x, W, V, W_2) = (\max(0, xW) \otimes xV)W_2$$

$$\mathrm{FFN}_{\mathrm{GEGLU}}(x, W, V, W_2) = (\mathrm{GELU}(xW) \otimes xV)W_2$$

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(x, W, V, W_2) = (\mathrm{Swish}_1(xW) \otimes xV)W_2$$

Модификации BERT

Что не так в классическом BERT?

- NSP (Next Sentence Prediction) – плохая задача предобучения
- Слишком маленькие и статичные данные - BERT обучался на сравнительно маленьких корпусах (BooksCorpus + Wikipedia, всего ~16 ГБ текста)
- Архитектурные ограничения - BERT использует "absolute positional encoding", что плохо масштабируется на длинные последовательности
- Статические (fixed) маски - При обучении BERT использует фиксированные позиции для маскированных токенов (в пределах одного батча).

RoBERTa

RoBERTa (Robustly Optimized BERT Approach) — это улучшенная версия классического BERT, предложенная Facebook AI в 2019 году.

Model	data	bsz	steps	SQuAD (v1.1/2.0)	MNLI-m	SST-2
RoBERTa						
with BOOKS + WIKI	16GB	8K	100K	93.6/87.3	89.0	95.3
+ additional data (§3.2)	160GB	8K	100K	94.0/87.7	89.3	95.6
+ pretrain longer	160GB	8K	300K	94.4/88.7	90.0	96.1
+ pretrain even longer	160GB	8K	500K	**94.6/89.4**	**90.2**	**96.4**
$\text{BERT}_{\text{LARGE}}$						
with BOOKS + WIKI	13GB	256	1M	90.9/81.8	86.6	93.7
$\text{XLNet}_{\text{LARGE}}$						
with BOOKS + WIKI	13GB	256	1M	94.0/87.8	88.4	94.4
+ additional data	126GB	2K	500K	94.5/88.8	89.8	95.6

Table 4: Development set results for RoBERTa as we pretrain over more data (16GB → 160GB of text) and pretrain for longer (100K → 300K → 500K steps). Each row accumulates improvements from the rows above. RoBERTa matches the architecture and training objective of $\text{BERT}_{\text{LARGE}}$. Results for $\text{BERT}_{\text{LARGE}}$ and $\text{XLNet}_{\text{LARGE}}$ are from Devlin et al. (2019) and Yang et al. (2019), respectively. Complete results on all GLUE tasks can be found in the Appendix.

RoBERTa

Отличие	Подробности
Данные	RoBERTa обучали на более крупном корпусе (160 GB vs 16 GB в BERT). Источники: BookCorpus, CC-News, OpenWebText, Stories.
Эпохи и шаги	RoBERTa обучался дольше (больше шагов и оптимизаций), что позволяет лучше усвоить закономерности в тексте.
Batch size и learning rate	Использовали больший batch size (8192 против 256/512 у BERT) и tuned learning rate.
Удаление NSP (Next Sentence Prediction)	BERT учил NSP, но RoBERTa показала, что NSP не помогает → убрали этот предикт → лучше использует ресурсы на MLM.
Динамическая маска MLM	В BERT маски были фиксированные для каждого токена, в RoBERTa маски генерируются случайно на каждой эпохе → модель видит больше вариаций.
Training tricks	Использование longer sequences, more dropout tuning, better optimizer hyperparameters.

ELECTRA

BERT обучается на задаче Masked Language Modeling (MLM): часть токенов в предложении заменяется на `[MASK]`, и модель пытается предсказать эти токены. Однако `[MASK]`-токены не встречаются в реальных данных, и это создаёт разрыв между обучением и инференсом.

ELECTRA предлагает другой подход - Replace Token Detection:

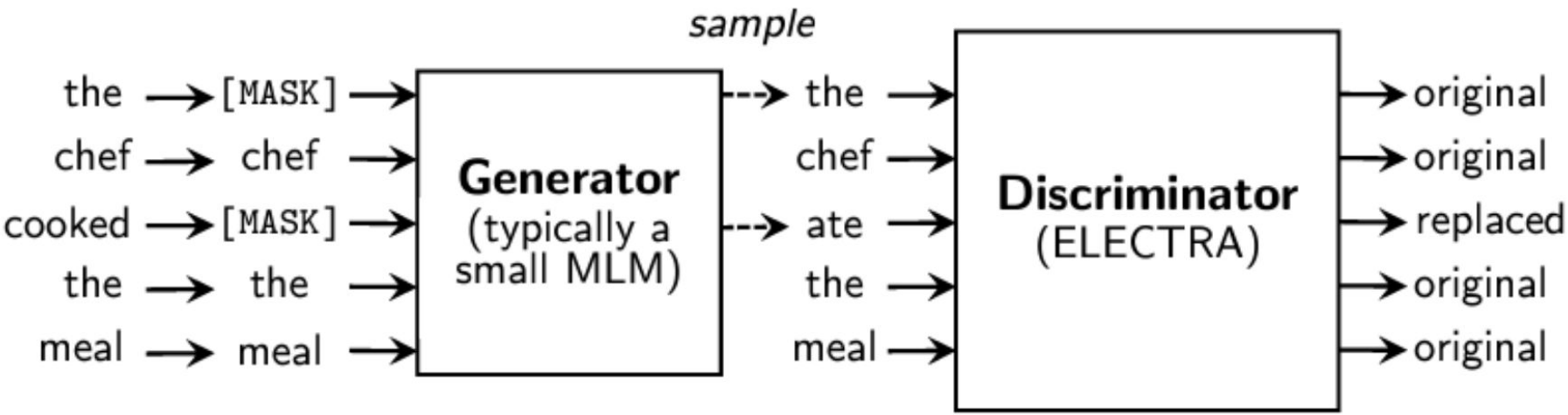
1. Вместо того чтобы предсказывать маскированные токены, ELECTRA обучается различать настоящие токены от "поддельных".

2. Для генерации "поддельных" токенов используется маленький MLM-генератор (как BERT), который заменяет случайные токены в тексте на предсказанные.

3. Основная модель (дискриминатор) учится предсказывать, какие токены настоящие, а какие подменены.

ELECTRA

Эмбеддинги: в последнем слое дискриминатора перед softmax



BART

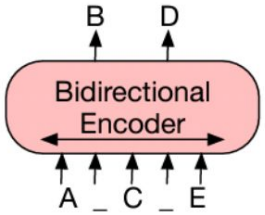
BART (Bidirectional and Auto-Regressive Transformers) — это трансформерная модель, разработанная Facebook AI, которая объединяет лучшие свойства двух подходов:

- Encoder-Decoder архитектура, как в классических моделях машинного перевода (например, в Transformer от Vaswani et al.).
- Совмещает bidirectional encoder (как в BERT) и autoregressive decoder (как в GPT).

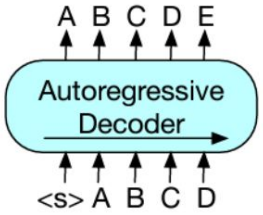
Основная идея

- BART обучается как автоэнкодер, который восстанавливает исходный текст из его искажённой версии.
- Для этого во время обучения вводятся различные типы шумов (удаление слов, замена, перестановка) — и модель учится восстанавливать оригинал.
- Это позволяет ей хорошо понимать контекст (encoder) и генерировать текст (decoder).

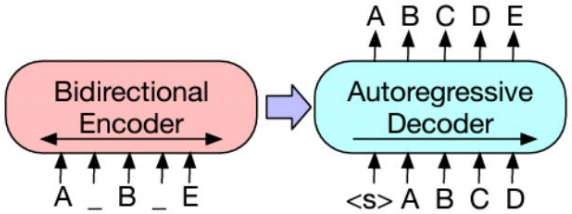
BART



(a) BERT: Random tokens are replaced with masks, and the document is encoded bidirectionally. Missing tokens are predicted independently, so BERT cannot easily be used for generation.



(b) GPT: Tokens are predicted auto-regressively, meaning GPT can be used for generation. However words can only condition on leftward context, so it cannot learn bidirectional interactions.



(c) BART: Inputs to the encoder need not be aligned with decoder outputs, allowing arbitary noise transformations. Here, a document has been corrupted by replacing spans of text with mask symbols. The corrupted document (left) is encoded with a bidirectional model, and then the likelihood of the original document (right) is calculated with an autoregressive decoder. For fine-tuning, an uncorrupted document is input to both the encoder and decoder, and we use representations from the final hidden state of the decoder.

Figure 1: A schematic comparison of BART with BERT (Devlin et al., 2019) and GPT (Radford et al., 2018).

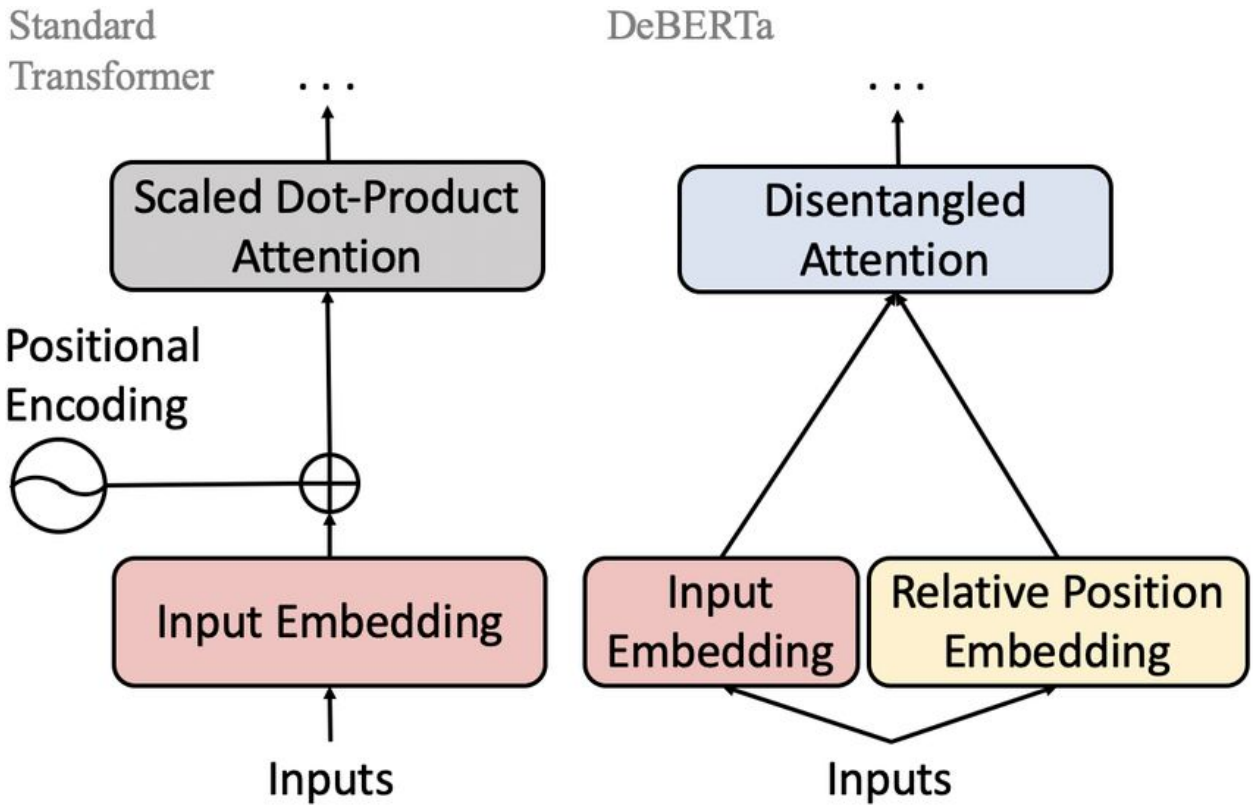
DeBERTa

DeBERTa — это улучшенная версия модели BERT, предложенная Microsoft Research, которая улучшает качество понимания текста за счёт двух ключевых новшеств:

1. Disentangled Attention (Разделённое внимание)
   В классическом self-attention у каждого токена есть только один позиционный эмбеддинг, который добавляется к содержательному (token) эмбеддингу.
   В DeBERTa позиционная информация и содержательная информация кодируются раздельно — это позволяет модели лучше различать токены по содержанию и позиции, улучшая понимание взаимосвязей в тексте.

2. Enhanced Mask Decoder
   Для задачи предсказания замаскированных токенов используется улучшенный декодер, который помогает эффективнее восстанавливать пропущенные слова.

DeBERTa


Standard Transformer
. . .
Scaled Dot-Product Attention
Positional Encoding
Input Embedding
Inputs
DeBERTa
. . .
Disentangled Attention
Input Embedding
Relative Position Embedding
Inputs

DeBERTa

Table 3: Comparison results on the GLUE development set.

Model	CoLA	QQP	MNLI-m/mm	SST-2	STS-B	QNLI	RTE	MRPC	Avg.
	Mcc	Acc	Acc	Acc	Corr	Acc	Acc	Acc	
#Train	8.5k	364k	393k	67k	7k	108k	2.5k	3.7k	
$\text{BERT}_{\text{large}}$	60.6	91.3	86.6/-	93.2	90.0	92.3	70.4	88.0	84.05
$\text{RoBERTa}_{\text{large}}$	68.0	92.2	90.2/90.2	96.4	92.4	93.9	86.6	90.9	88.82
$\text{XLNet}_{\text{large}}$	69.0	92.3	90.8/90.8	**97.0**	92.5	94.9	85.9	90.8	89.15
$\text{ELECTRA}_{\text{large}}$	69.1	92.4	90.9/-	96.9	92.6	95.0	88.0	90.8	89.46
$\text{DeBERTa}_{\text{large}}$	70.5	92.3	91.1/91.1	96.8	92.8	95.3	88.3	91.9	90.00
$\text{DeBERTaV3}_{\text{large}}$	**75.3**	**93.0**	**91.8/91.9**	96.9	**93.0**	**96.0**	**92.7**	**92.2**	**91.37**

Top-5 MTEB (12.03.2026)

Модель	Архитектура	Комментарий об архитектуре и позиции
KaLM-Embedding-Gemma3-12B-2511	Decoder-only (Gemma3-based)	Базируется на **Gemma-3** (decoder-only transformer), применяется как embedding через pooling последних скрытых слоёв, позиция учитывается через стандартный Transformer (RoPE/relative внутри decoder). (huggingface.co)
llama-embed-nemotron-8b	Transformer Decoder (bi-directional)	Основан на **Llama-3.1-8B decoder**, но встраивается в bi-encoder-подобную задачу (контрастное обучение); позиционные эмбеддинги унаследованы от Llama. (huggingface.co)
Qwen3-Embedding-8B	Encoder-like bi-encoder	Dense Transformer с контрастным обучением (на основе Qwen3), позиция кодируется в self-attention (обычно rotary); поддерживает Matryoshka dims. (GitHub)
gemini-embedding-001	**Encoder-only** ℹ️	Коммерческий текстовый эмбеддер от Gemini API, построен на собственном Transformer-базовом энкодере (bidirectional), принимает текст и выдаёт плотные вектора; позиция обрабатывается внутри attention (dense). (Google AI for Developers)
Qwen3-Embedding-4B	Encoder-like bi-encoder	Версия Qwen3-Embedding с меньшим количеством параметров по сравнению с 8B, та же архитектура с контрастным обучением и rotary/attention PE. (siliconflow.com)

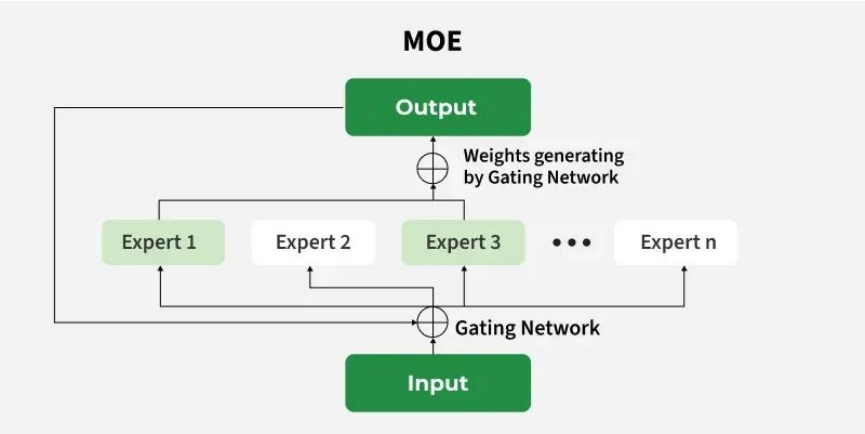
Mixture of Experts

MoE

В обычном Transformer каждый слой обрабатывает **все токены через все нейроны (все параметры)**.

В MoE:

- Каждый слой содержит **несколько "экспертов" (experts)** — это отдельные подсети, например маленькие feed-forward блоки.
- Для каждого входного токена выбирается **только один или несколько экспертов**, через **gating-механизм** (router).
- То есть **не все параметры участвуют в каждом проходе**, а только выбранные эксперты.

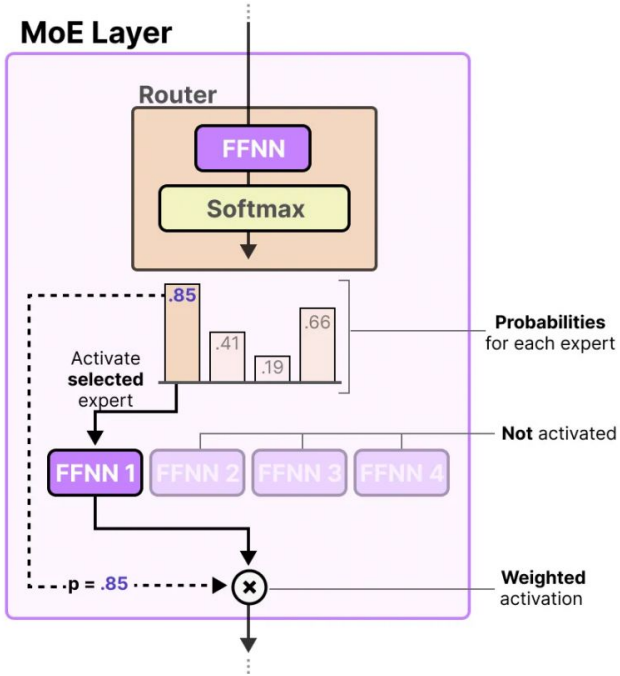


Внезапная история MoE

Год появления	Событие
1991	Первая формулировка Mixture of Experts в работе *“Adaptive Mixtures of Local Experts”* (Jacobs, Jordan, Nowlan, Hinton).
1994	Развитие идеи — Hierarchical Mixture of Experts (HME) (Jordan & Jacobs).
2017	Появление Sparsely-Gated MoE — первое масштабное применение MoE в глубоких нейросетях (Google).
2021	Использование в больших языковых моделях: Switch Transformer.
2024+	Широкое распространение MoE в LLM: Mixtral, DeepSeek-MoE и др.

Router в MoE

Router в Mixture of Experts (MoE) — это маленькая нейросеть, которая решает, каким экспертам отправить каждый токен. Обычно это один линейный слой + softmax.



Router input

Router получает **скрытое состояние токена** из Transformer слоя:

$$h \in \mathbb{R}^d$$

где

- h — embedding токена после attention
- d — размер скрытого пространства.

То есть router смотрит **на смысл токена**, уже обработанный attention.

Router

Router — это линейная проекция:

$$g = W_r h$$

где

- $W_r \in \mathbb{R}^{N \times d}$
- N — количество экспертов.

На выходе получается:

$$g = (g_1, g_2, \ldots, g_N)$$

Это **score каждого эксперта** для данного токена.

Обычно применяется softmax:

$$p_i = \frac{e^{g_i}}{\sum_j e^{g_j}}$$

Router output

Каждый выбранный эксперт обрабатывает токен:

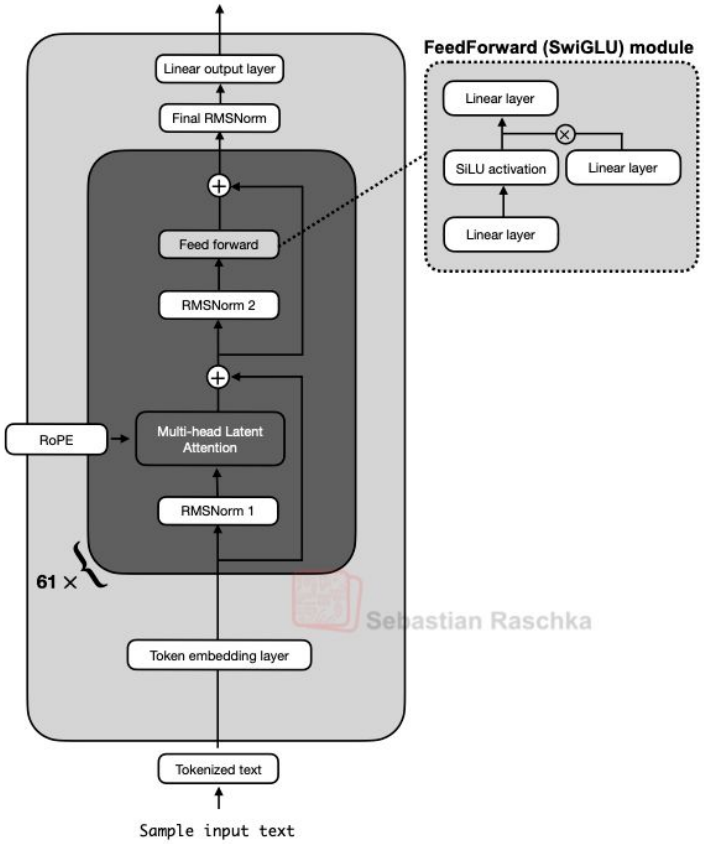
$$y_i = expert_i(h)$$

Потом outputs комбинируются:

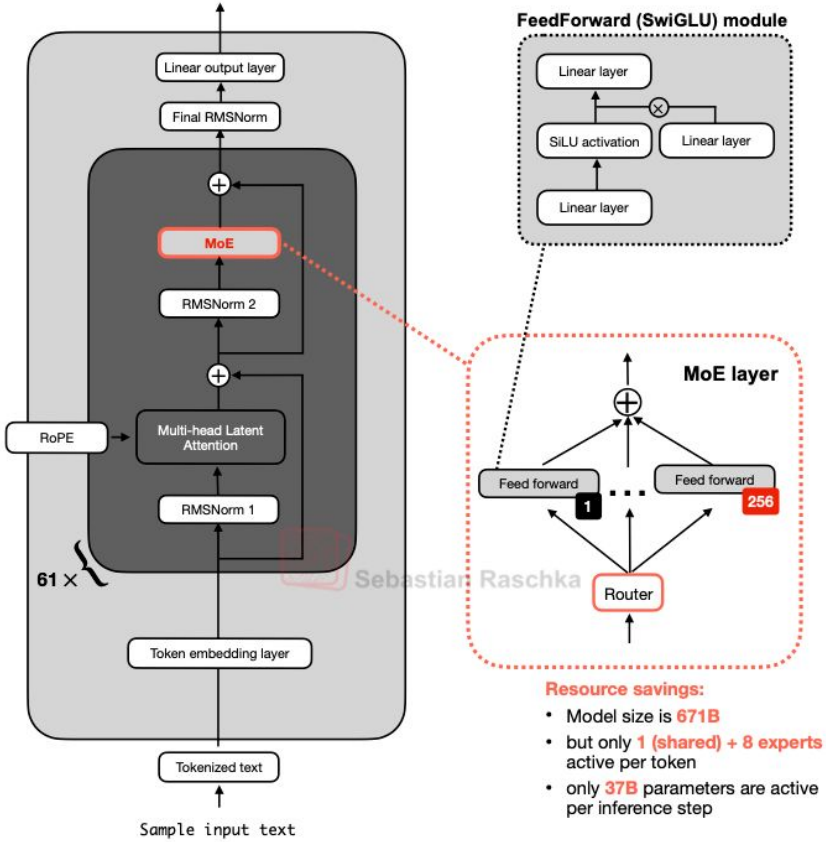
$$y = \sum_{i \in top-k} p_i \cdot y_i$$

То есть выход — **взвешенная сумма экспертов**.

Architecture without MoE (“dense”)



DeepSeek V3/R1 with MoE (“sparse”)



Resource savings:

- Model size is 671B
- but only 1 (shared) + 8 experts active per token
- only 37B parameters are active per inference step

MoE

- **Масштабируемость параметров** — модель может иметь намного больше параметров, чем dense-модель при сопоставимых вычислениях.
- **Меньше вычислений на токен** — для каждого токена активируется только **top-k экспертов**, а не вся сеть.
- **Специализация экспертов** — разные эксперты могут обучаться на разных типах данных (языки, код, математика и т. д.).
- **Лучшее соотношение качество / compute** — часто достигается более высокое качество при тех же FLOPs.
- **Эффективность для больших моделей** — позволяет строить модели с сотнями миллиардов или триллионами параметров.

Но! Память под параметры — не экономится.

Все эксперты должны быть загружены в память (GPU/CPU), поэтому MoE-модель часто значительно больше по VRAM, чем dense-модель.

MoE в коде

https://colab.research.google.com/drive/1eJGnxO7Y1EW4ZTM1KRJEY6j9IiOtK7aU?usp=sharing